

Week 3 — Day 17 Manual (WSL)

Guardrails: Confidence, Refusal or Clarify, and Logging to Postgres

Extension of Day 16: In the original warmup plan, Day 17 is guardrails: confidence, refusal/clarify, and logging to Postgres. This comes after Roman-Urdu normalization and dual-query retrieval, and before agent loop work.

1. Purpose of Day 17

By Day 16, you should already have:

- a working RAG endpoint
- retrieval from Qdrant
- Roman-Urdu normalization
- dual-query retrieval with merged results

That is still not a safe enough system.

A weak RAG system does two bad things very quickly:

- it answers confidently when retrieval is weak
- it gives vague or misleading responses when the evidence is poor

That is the point of Day 17. You will add guardrails so the system can decide whether it should:

- answer normally
- refuse because evidence is too weak
- ask for clarification because the query is too vague

You will also log retrieval and answer traces to Postgres so the system becomes inspectable instead of opaque.

2. Day 17 Scope

Today, you will add four practical controls:

1. Confidence estimation from retrieval results
2. Refusal behavior when evidence is too weak
3. Clarification behavior when the question is underspecified
4. Logging of queries, retrieval decisions, and answers to Postgres

This is not full production safety. It is the first real control layer.

3. Outcomes and Deliverables

By the end of Day 17, your repository must contain:

- a guardrails module with confidence and decision logic
- clear branching between:
 - answer
 - refuse
 - clarify
- Postgres logging for RAG requests and outcomes
- an updated /rag response that exposes the decision outcome
- screenshots stored under:

docs/screenshots/week3-day17/

- a pull request merged into main

4. Preconditions

Before starting Day 17, the following must already work:

- your FastAPI app runs
- your Day 15 RAG endpoint works
- your Day 16 normalized dual-query retrieval works
- Postgres is reachable through your current Docker Compose stack

- SQLAlchemy and Alembic are already integrated from earlier days
- your repository structure is still clean

If retrieval is unreliable, then your confidence logic will be garbage. Bad retrieval plus fancy rules is still bad retrieval.

5. Start in WSL

From Windows PowerShell:

```
wsl ~
```

In WSL:

```
cd ~/<your-repo>
source .venv/bin/activate
python -m pip install --upgrade pip
```

Create a Day 17 branch:

```
git checkout main
git pull
git checkout -b feature/week3-day17-guardrails-logging
```

6. Why Guardrails Are Needed

A retrieval pipeline returning something is not the same as a retrieval pipeline returning enough.

Typical failure cases:

- the top result is weak but the model still answers anyway
- the question is too vague, but the system pretends it knows what the user meant
- the query retrieves loosely related chunks and the model fills gaps with invention
- there is no visibility into what happened after the answer is returned

Day 17 exists to stop those failures from being invisible.

7. Day 17 Design Strategy

Use simple, explicit rules. Do not fake sophistication.

For every RAG request:

1. retrieve results
2. inspect retrieval quality
3. compute a simple confidence signal
4. choose one action:
 - a. answer
 - b. clarify
 - c. refuse
5. if action is answer, continue to LLM generation
6. if action is clarify or refuse, return a controlled response
7. log the full trace to Postgres

This is the right baseline.

8. Update the Repository Structure

By the end of Day 17, your structure should resemble:

```
README.md
Dockerfile
docker-compose.yml
pyproject.toml
.gitignore
.env.example
src/
  <package_name>/
    __init__.py
    api.py
    settings.py
    db.py
```

```
models.py
vector_store.py
retrieval.py
rag.py
llm_client.py
normalization.py
guardrails.py
schema.py
tests/
scripts/
alembic/
docs/
screenshots/
  week3-day15/
  week3-day16/
  week3-day17/
```

Notes

- guardrails.py should contain decision logic
- models.py should include a log table if it does not already
- rag.py should orchestrate the full control flow
- api.py should remain thin

9. Add Guardrail Settings

Open:

```
src/<package_name>/settings.py
```

Add fields like these to your settings class:

```
from pydantic import Field
```

```
default_search_limit: int = Field(default=3)
merged_search_limit: int = Field(default=5)
```

```
min_top_score_for_answer: float = Field(default=0.55,  
validation_alias="MIN_TOP_SCORE_FOR_ANSWER")  
min_avg_score_for_answer: float = Field(default=0.45,  
validation_alias="MIN_AVG_SCORE_FOR_ANSWER")  
min_results_for_answer: int = Field(default=2,  
validation_alias="MIN_RESULTS_FOR_ANSWER")
```

```
enable_clarify_behavior: bool = Field(default=True,  
validation_alias="ENABLE_CLARIFY_BEHAVIOR")  
enable_refuse_behavior: bool = Field(default=True,  
validation_alias="ENABLE_REFUSE_BEHAVIOR")  
enable_rag_logging: bool = Field(default=True,  
validation_alias="ENABLE_RAG_LOGGING")
```

Update .env.example:

```
MIN_TOP_SCORE_FOR_ANSWER=0.55  
MIN_AVG_SCORE_FOR_ANSWER=0.45  
MIN_RESULTS_FOR_ANSWER=2  
ENABLE_CLARIFY_BEHAVIOR=true  
ENABLE_REFUSE_BEHAVIOR=true  
ENABLE_RAG_LOGGING=true
```

Notes

- these thresholds are starting points, not magic values
- if your retrieval scores are structurally weak, lowering thresholds blindly is just self-deception

10. Create the Guardrails Module

Create:

```
src/<package_name>/guardrails.py
```

Paste:

```
from __future__ import annotations
```

```
from statistics import mean
```

```
from <package_name>.settings import settings
```

```
def is_query_too_vague(question: str) -> bool:  
    stripped = question.strip().lower()
```

```
    vague_queries = {  
        "tell me",  
        "explain",  
        "what is this",  
        "help",  
        "answer this",  
        "maloomat do",  
    }
```

```
    if stripped in vague_queries:  
        return True
```

```
    if len(stripped.split()) <= 1:  
        return True
```

```
    return False
```

```
def compute_confidence(results: list[dict]) -> dict:
```

```
    if not results:  
        return {  
            "top_score": 0.0,  
            "avg_score": 0.0,  
            "result_count": 0,  
        }
```

```
    scores = [item["score"] for item in results]
```

```
    return {  
        "top_score": max(scores),  
        "avg_score": mean(scores),
```

```
    "result_count": len(results),  
}
```

```
def choose_rag_action(question: str, results: list[dict]) -> dict:  
    confidence = compute_confidence(results)
```

```
    if settings.enable_clarify_behavior and is_query_too_vague(question):  
        return {  
            "action": "clarify",  
            "reason": "Question is too vague or underspecified.",  
            "confidence": confidence,  
        }
```

```
    if settings.enable_refuse_behavior:  
        if confidence["result_count"] < settings.min_results_for_answer:  
            return {  
                "action": "refuse",  
                "reason": "Not enough supporting results were retrieved.",  
                "confidence": confidence,  
            }
```

```
        if confidence["top_score"] < settings.min_top_score_for_answer:  
            return {  
                "action": "refuse",  
                "reason": "Top retrieval score is too weak.",  
                "confidence": confidence,  
            }
```

```
        if confidence["avg_score"] < settings.min_avg_score_for_answer:  
            return {  
                "action": "refuse",  
                "reason": "Average retrieval confidence is too weak.",  
                "confidence": confidence,  
            }
```

```
    return {  
        "action": "answer",  
        "reason": "Retrieved context is sufficient for answer generation.",  
        "confidence": confidence,  
    }
```



```
}
```

Important: Replace `<package_name>` with your real package name.

11. Add a Logging Table in Postgres

You need a table to log what happened for each RAG call.

Open:

```
src/<package_name>/models.py
```

Add a model like this:

```
from __future__ import annotations
```

```
from datetime import datetime
```

```
from sqlalchemy import DateTime, Float, Integer, String, Text
from sqlalchemy.orm import Mapped, mapped_column
```

```
from <package_name>.db import Base
```

```
class RagLog(Base):
```

```
    __tablename__ = "rag_logs"
```

```
    id: Mapped[int] = mapped_column(Integer, primary_key=True, index=True)
```

```
    question: Mapped[str] = mapped_column(Text, nullable=False)
```

```
    normalized_query: Mapped[str] = mapped_column(Text, nullable=False)
```

```
    action: Mapped[str] = mapped_column(String(32), nullable=False)
```

```
    reason: Mapped[str] = mapped_column(Text, nullable=False)
```

```
    answer: Mapped[str] = mapped_column(Text, nullable=False, default="")
```

```
    top_score: Mapped[float] = mapped_column(Float, nullable=False, default=0.0)
```

```
    avg_score: Mapped[float] = mapped_column(Float, nullable=False, default=0.0)
```

```
    result_count: Mapped[int] = mapped_column(Integer, nullable=False, default=0)
```

```
    created_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow,
```

nullable=False)

Replace <package_name> with the real one.

12. Create an Alembic Migration

Create a migration for the new rag_logs table.

Run:

```
alembic revision --autogenerate -m "add rag_logs table"
```

Then apply it:

```
alembic upgrade head
```

Checkpoint

At this point:

- the migration file exists
- the rag_logs table exists in Postgres
- your schema is in sync with your models

If Alembic autogenerate fails, your DB setup is still shaky.

13. Add a Logging Helper

Create:

```
src/<package_name>/logging_utils.py
```

Paste:

```
from __future__ import annotations

from sqlalchemy.orm import Session

from <package_name>.models import RagLog
```

```
def create_rag_log(
    db: Session,
    *,
    question: str,
    normalized_query: str,
    action: str,
    reason: str,
    answer: str,
    top_score: float,
    avg_score: float,
    result_count: int,
) -> RagLog:
    log = RagLog(
        question=question,
        normalized_query=normalized_query,
        action=action,
        reason=reason,
        answer=answer,
        top_score=top_score,
        avg_score=avg_score,
        result_count=result_count,
    )
    db.add(log)
    db.commit()
    db.refresh(log)
    return log
```

This should stay separate from route code.

14. Update the RAG Module

Open:

```
src/<package_name>/rag.py
```

Refactor the orchestration so it uses guardrails before generating an answer.

A practical version is:

```
from __future__ import annotations

from <package_name>.guardrails import choose_rag_action
from <package_name>.llm_client import generate_answer_from_prompt
from <package_name>.retrieval import dual_query_search
```

```
def build_context_block(results: list[dict]) -> str:
    parts: list[str] = []
```

```
    for idx, item in enumerate(results, start=1):
```

```
        parts.append(
            f"[Source {idx}]\n"
            f"doc_id: {item['doc_id']}\n"
            f"chunk_id: {item['chunk_id']}\n"
            f"title: {item['title']}\n"
            f"text: {item['text']}\n"
        )
```

```
    return "\n".join(parts)
```

```
def build_rag_prompt(question: str, results: list[dict]) -> str:
    context = build_context_block(results)
```

```
    return f"""
```

Answer the question using only the context below.

Question:

```
{question}
```

Context:

{context}

Rules:

- Use only the provided context.
- Do not invent facts.
- If the answer is not supported by the context, say that clearly.
- Mention supporting sources using the source numbers.

"".strip()

```
def answer_with_rag(question: str, limit: int) -> dict:
```

```
    retrieval_payload = dual_query_search(question, limit)
```

```
    results = retrieval_payload["results"]
```

```
    normalized_query = retrieval_payload["normalized_query"]
```

```
    decision = choose_rag_action(question, results)
```

```
    if decision["action"] == "clarify":
```

```
        return {
```

```
            "question": question,
```

```
            "normalized_query": normalized_query,
```

```
            "action": "clarify",
```

```
            "reason": decision["reason"],
```

```
            "answer": "Please clarify your question with more specific details.",
```

```
            "sources": results,
```

```
            "confidence": decision["confidence"],
```

```
        }
```

```
    if decision["action"] == "refuse":
```

```
        return {
```

```
            "question": question,
```

```
            "normalized_query": normalized_query,
```

```
            "action": "refuse",
```

```
            "reason": decision["reason"],
```

```
            "answer": "I do not have enough reliable context to answer that safely.",
```

```
            "sources": results,
```

```
            "confidence": decision["confidence"],
```

```
        }
```

```

prompt = build_rag_prompt(question, results)
answer = generate_answer_from_prompt(prompt)

return {
    "question": question,
    "normalized_query": normalized_query,
    "action": "answer",
    "reason": decision["reason"],
    "answer": answer,
    "sources": results,
    "confidence": decision["confidence"],
}

```

15. Update the Response Schemas

Open:

```
src/<package_name>/schema.py
```

Extend the RAG response schema:

```
from __future__ import annotations
```

```
from pydantic import BaseModel, Field
```

```

class RagRequest(BaseModel):
    question: str = Field(min_length=1, description="The user question.")
    limit: int = Field(default=3, ge=1, le=10)

```

```

class RagSourceItem(BaseModel):
    score: float
    doc_id: str
    chunk_id: str
    title: str

```

```
language: str
source: str
chunk_index: int
text: str
```

```
class RagConfidence(BaseModel):
    top_score: float
    avg_score: float
    result_count: int
```

```
class RagResponse(BaseModel):
    question: str
    normalized_query: str
    action: str
    reason: str
    answer: str
    confidence: RagConfidence
    sources: list[RagSourceItem]
```

Why this matters

You need visibility into:

- what the system decided
- why it decided that
- how strong the retrieval looked

Otherwise Day 17 is just hidden logic with no inspectability.

16. Update the API Route and Log to Postgres

Open:

```
src/<package_name>/api.py
```

Update the /rag route so it writes logs to Postgres.

Example:

```
from __future__ import annotations

from fastapi import Depends, FastAPI, HTTPException
from sqlalchemy.orm import Session

from <package_name>.db import get_db
from <package_name>.logging_utils import create_rag_log
from <package_name>.rag import answer_with_rag
from <package_name>.schema import RagRequest, RagResponse
from <package_name>.settings import settings

app = FastAPI(title="AI Retrieval and RAG API")

@app.post("/rag", response_model=RagResponse)
def rag_endpoint(payload: RagRequest, db: Session = Depends(get_db)) -> RagResponse:
    try:
        result = answer_with_rag(payload.question, payload.limit)

        if settings.enable_rag_logging:
            confidence = result["confidence"]
            create_rag_log(
                db,
                question=result["question"],
                normalized_query=result["normalized_query"],
                action=result["action"],
                reason=result["reason"],
                answer=result["answer"],
                top_score=confidence["top_score"],
                avg_score=confidence["avg_score"],
                result_count=confidence["result_count"],
            )

        return RagResponse(
            question=result["question"],
            normalized_query=result["normalized_query"],
            action=result["action"],
```



```

        reason=result["reason"],
        answer=result["answer"],
        confidence=result["confidence"],
        sources=result["sources"],
    )
except Exception as exc:
    raise HTTPException(status_code=500, detail=f"RAG failed: {exc}") from exc

```

Replace <package_name> with your actual package name.

17. Add a Log Inspection Script

Create:

scripts/day17_show_rag_logs.py

Paste:

```

from __future__ import annotations

from sqlalchemy import select

from <package_name>.db import SessionLocal
from <package_name>.models import RagLog


def main() -> None:
    with SessionLocal() as session:
        stmt = select(RagLog).order_by(RagLog.created_at.desc()).limit(10)
        rows = session.execute(stmt).scalars().all()

        for row in rows:
            print("=" * 80)
            print(f'id: {row.id}')
            print(f'question: {row.question}')
            print(f'normalized_query: {row.normalized_query}')
            print(f'action: {row.action}')

```

```
print(f'reason: {row.reason} ")
print(f'top_score: {row.top_score} ")
print(f'avg_score: {row.avg_score} ")
print(f'result_count: {row.result_count} ")
print(f'created_at: {row.created_at} ")
print(f'answer: {row.answer[:200]} ")
```

Run with:

```
export PYTHONPATH=src
python scripts/day17_show_rag_logs.py
```

18. Run the API

```
export PYTHONPATH=src
fastapi dev src/<package_name>/api.py
```

Or:

```
export PYTHONPATH=src
uvicorn <package_name>.api:app --reload
```

19. Test the Three Behaviors

Open Swagger UI:

<http://127.0.0.1:8000/docs>

19.1 Normal answer case

```
{
  "question": "What does Qdrant do in this system?",
  "limit": 3
```

```
}
```

Expected: action = "answer"

19.2 Clarify case

```
{  
  "question": "help",  
  "limit": 3  
}
```

Expected: action = "clarify"

19.3 Refuse case

```
{  
  "question": "Give me details about a topic that does not exist in the dataset",  
  "limit": 3  
}
```

Expected: action = "refuse" if retrieval is weak enough

19.4 Check logs

After several requests, run:

```
export PYTHONPATH=src  
python scripts/day17_show_rag_logs.py
```

You should see records for answer, clarify, and refuse paths.

20. Add Basic Tests

Create:

tests/test_day17_guardrails.py

Paste:

```
from <package_name>.guardrails import choose_rag_action, is_query_too_vague
```

```
def test_is_query_too_vague_for_single_word() -> None:
    assert is_query_too_vague("help") is True
```

```
def test_choose_rag_action_returns_refuse_for_empty_results() -> None:
    decision = choose_rag_action("what is this", [])
    assert decision["action"] in {"clarify", "refuse"}
```

```
def test_choose_rag_action_returns_answer_for_strong_results() -> None:
    results = [
        {"score": 0.90, "chunk_id": "c1"},
        {"score": 0.82, "chunk_id": "c2"},
        {"score": 0.80, "chunk_id": "c3"},
    ]
    decision = choose_rag_action("What does Qdrant do?", results)
    assert decision["action"] == "answer"
```

Replace <package_name> with the actual one.

Note

These are baseline tests, not enough for a serious system. But without even these, your control logic is basically unverified.

21. Create the Evidence Folder

Create:

```
mkdir -p docs/screenshots/week3-day17
touch docs/screenshots/week3-day17/.gitkeep
```

Required screenshots:

1. 01-guardrails-code.png

guardrails.py open in the editor

2. 02-answer-case.png

/rag response showing action: answer

3. 03-clarify-case.png

/rag response showing action: clarify

4. 04-refuse-case.png

/rag response showing action: refuse

5. 05-rag-logs-table.png

Postgres table or terminal proof that rag_logs exists

6. 06-log-inspection-script.png

terminal output from day17_show_rag_logs.py

7. 07-pytest-pass.png

terminal output showing Day 17 tests pass

8. 08-git-status-clean.png

terminal output showing only intended tracked files

22. Update the Root README

Your README.md must now reflect Day 17 progress.

Update these sections:

Current Scope

Add that the project now supports:

- confidence-based decision control
- refusal when retrieval is too weak
- clarification when the question is underspecified
- Postgres logging of RAG request traces

Repository Structure

Include:

- `guardrails.py`
- `logging_utils.py`

Progress by Day

Update to include:

- Day 15: RAG endpoint v1
- Day 16: Roman-Urdu normalization and dual-query retrieval
- Day 17: guardrails, refusal/clarify behavior, and logging to Postgres

Next Steps

Your next steps should now point toward:

- agent loop design
- tool selection rules
- response verification
- production hardening

This follows the original sequence, where Day 18 is the agent loop.

23. Git Workflow

Stage Files

```
git add \  
  README.md \  
  .env.example \  
  alembic/ \  
  src/ \  
  tests/ \  
  scripts/ \  
  docs/screenshots/week3-day17/.gitkeep
```

Commit

```
git commit -m "Add Day 17 guardrails and Postgres logging"
```

Push

```
git push -u origin feature/week3-day17-guardrails-logging
```

Open a Pull Request

- Base: main
- Compare: feature/week3-day17-guardrails-logging

PR Title:

Day 17: Guardrails with confidence, refusal/clarify, and Postgres logging

PR Description must include:

- how to run the API
- how to test answer, clarify, and refuse cases
- how to inspect logs:

```
export PYTHONPATH=src
python scripts/day17_show_rag_logs.py
```

- evidence path:

docs/screenshots/week3-day17/

24. Completion Checklist

Before marking Day 17 complete, confirm all of the following:

- confidence logic exists
- clarify behavior exists for vague queries
- refuse behavior exists for weak retrieval
- the /rag response exposes action and reason
- RAG logs are written to Postgres
- the log table exists through Alembic migration
- answer, clarify, and refuse flows all work
- screenshots are stored under docs/screenshots/week3-day17/
- the root README.md reflects Day 17 progress
- the pull request has been merged into main

25. Practical Notes

Keep these points in mind:

- Thresholds are not truth. They are rough controls. You will need tuning.
- If the system refuses too often, either your thresholds are too strict or your retrieval is weak.
- If the system answers too often on garbage retrieval, your thresholds are too lax.
- Clarify behavior should be used for vague questions, not as a lazy fallback for everything.
- Logging is not optional if you want to debug real failures. Without logs, you are guessing.

26. What Day 17 Prepares You For

After completing Day 17, you should be ready for:

- an agent loop that plans actions explicitly
- controlled tool usage
- stronger verification before final responses
- production-style flow control

That matches the original plan, because Day 18 is the agent loop after guardrails and logging.